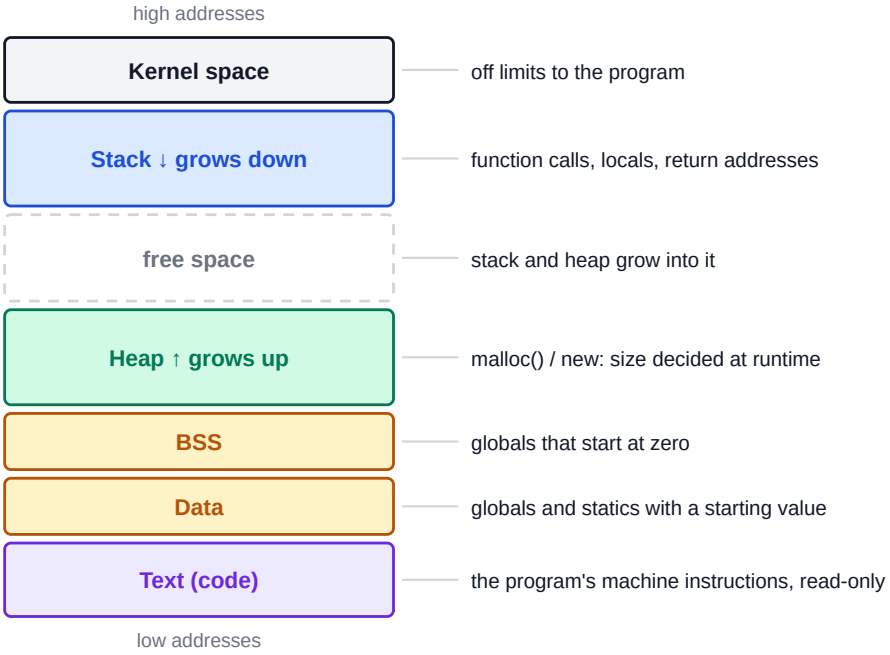
OPERATING SYSTEMS & SECURITY · MODULE 2

Processes, Threads & Scheduling

Lab cheat sheet: how a program runs in memory, how the OS juggles processes and threads, how it decides who runs next, and where the security problems hide. Commands marked "tested" were run on a Linux box. The scheduling example uses made-up numbers.

1. A program in memory becomes a process



See it live:

```
cat /proc/<PID>/maps
pmap -x <PID>
size ./program
```

One process's virtual address space. A **program** is a file on disk. Once it's loaded into memory and running, it's a **process**.

SEE THE SECTIONS IN A REAL PROGRAM

```
; add.asm: a tiny 64-bit Linux program (NASM syntax)
section .data              ; DATA segment: variables with starting values
    a    dq 7
    b    dq 5
    sum  dq 0

section .text              ; TEXT segment: the instructions
    global _start
_start:                    ; entry point
    mov rax, [a]           ; load a into register RAX
    add rax, [b]           ; RAX = 7 + 5
    mov [sum], rax         ; store the result back in memory
    mov rdi, rax           ; exit status = 12
    mov rax, 60            ; system call number 60 = exit
    syscall                ; ask the kernel to end the process
```

BUILD AND RUN IT (TESTED)

`nasm -f elf64 add.asm -o add.o`	assemble (install `nasm`)
`ld add.o -o add`	link into a program
`./add; echo $?`	run it; prints **12** (the exit status)

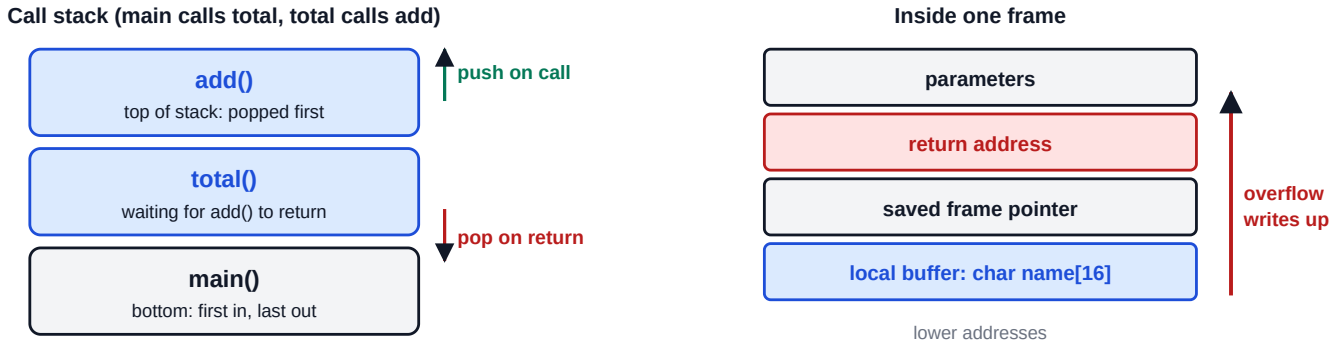
INSPECT IT (TESTED)

`size add`	bytes in text, data and bss
`objdump -d add`	disassemble the text section
`strace ./add`	shows the single `exit(12)` system call

`file add`	ELF 64-bit, x86-64

The same layout applies to every program you run: `pmap -x $$` shows it for your own shell.

2. The stack: push, pop, LIFO



Copy 40 bytes into a 16-byte buffer and the extra bytes overwrite the return address: the classic stack buffer overflow.

Each function call pushes a **frame** (parameters, return address, locals). Returning pops it. Last in, first out, like a pile of plates.

	Stack	Heap
Managed by	automatic: push on call, pop on return	the program: `malloc` / `free` (C), garbage collector (Python, Java)
Speed and size	very fast, small and fixed (often 8 MB)	slower, can grow large
Holds	locals, parameters, return addresses	data whose size is only known at runtime
Classic bugs	stack overflow (runaway recursion), stack buffer overflow	memory leaks, use-after-free, heap overflow

CHECK

`ulimit -s`	max stack size in KB (8192 = 8 MB)
`cat /proc/sys/kernel/randomize_va_space`	ASLR: 2 = full (stack, heap, libraries move each run)

LOOK INSIDE WITH GDB

`gdb ./prog then bt`	backtrace: print the current call stack
`gdb then info frame`	show the return address saved in a frame

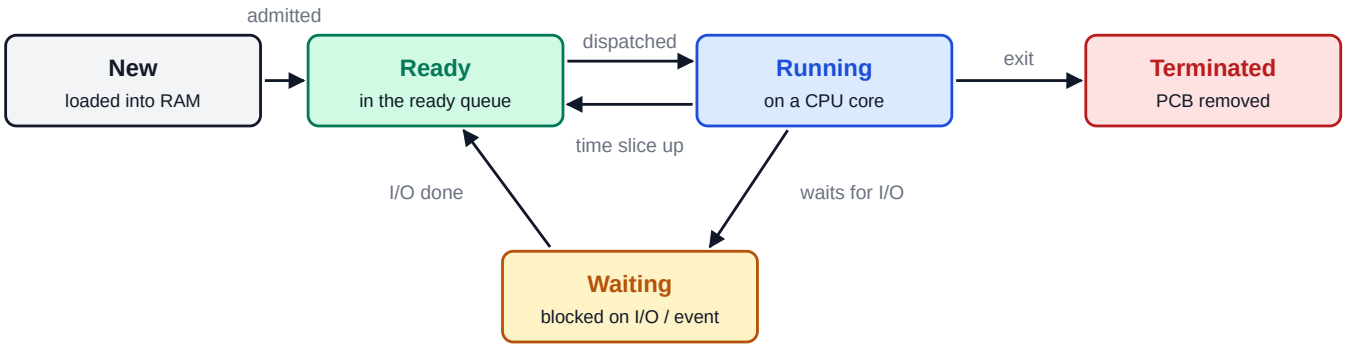
3. 32-bit vs 64-bit

	32-bit	64-bit
Data per instruction	32 bits (4 bytes)	64 bits (8 bytes)
Address space per process	4 GB (2^{32} bytes)	16 EB in theory (2^{64}); real CPUs use 48-bit (256 TB) or 57-bit addresses
Runs the other's software?	can't run 64-bit programs	runs 32-bit programs if 32-bit libraries are installed
Security	ASLR is weak: few possible positions to guess	ASLR is much stronger; NX/DEP on by default
Linux name	`i386`, `i686`, `armhf`	`x86_64` (= amd64), `aarch64` (= arm64)

`uname -m`	CPU/kernel architecture
`getconf LONG_BIT`	32 or 64
`file /bin/ls`	is a program 32-bit or 64-bit ELF?
`lscpu | grep 'op-mode'`	which modes the CPU supports

4. Process life cycle

Linux STAT letters: R = ready or running S / D = waiting T = stopped Z = zombie (exited, parent hasn't collected it)



A process moves between states many times a second. Only one process per core can be **Running** at any instant.

SEE STATES

`ps -eo pid,ppid,stat,cmd`	every process with its state letter
`grep State /proc/<PID>/status`	one process's state
`ps -eo stat | sort | uniq -c`	how many processes in each state

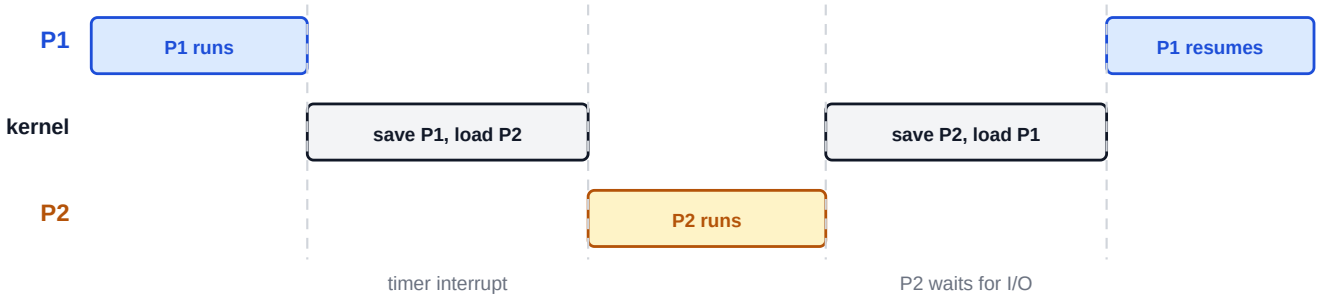
TRY IT

`sleep 300 &`	start a process that sits in S (sleeping)
`kill -STOP %1 / kill -CONT %1`	push it to T (stopped) and back
`ps -eo pid,ppid,stat,cmd | awk '$3 ~ /Z/'`	find zombies (kill or fix the parent)

5. Process control block and context switching

PCB for PID 2041		
Process ID	2041 (parent 1880)	`/proc/2041/status (Pid, PPid)`
State	Running	`ps -o stat -p 2041`
Program counter	next instruction	`saved on every context switch`
CPU registers	rax, rsp, rip ...	`saved on every context switch`
Scheduling info	priority, nice 0	`ps -o ni,pri,cls -p 2041`
Memory info	page tables, limits	`/proc/2041/maps, /proc/2041/limits`
Accounting	CPU time used	`ps -o etime,time -p 2041`
I/O status	open files, sockets	`ls -l /proc/2041/fd`

The **PCB** is the kernel's record of one process. Linux calls it `task_struct`; `/proc/<PID>/` lets you read most of it.



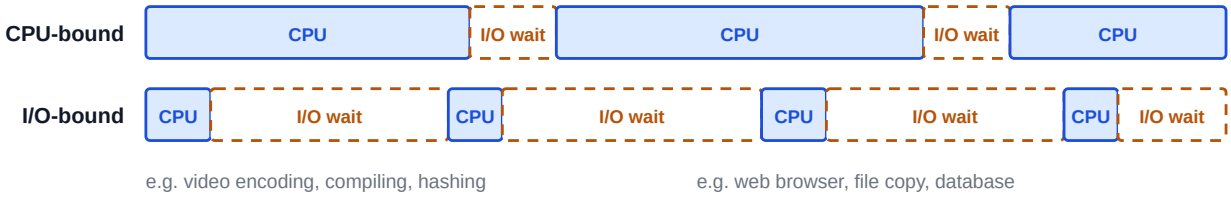
Grey blocks are overhead: the CPU does no useful work for either program while it switches.

A **context switch**: the kernel saves the running process's registers and program counter into its PCB, then loads another PCB and resumes that process.

MEASURE SWITCHING

`vmstat 1 5`	`cs` column = context switches per second
`grep ctxt /proc/<PID>/status`	voluntary (it waited) vs nonvoluntary (time slice ran out)
`pidstat -w 1`	switches per process (install `sysstat`)
`cat /proc/<PID>/sched`	per-process scheduler stats (if the kernel exposes it)

6. CPU bursts, I/O bursts, spooling



While one process waits on I/O, the scheduler hands the CPU to another. That overlap is how multitasking stays fast.

A **CPU burst** is a stretch of computing. An **I/O burst** is time spent waiting for a device. Most programs alternate between them.

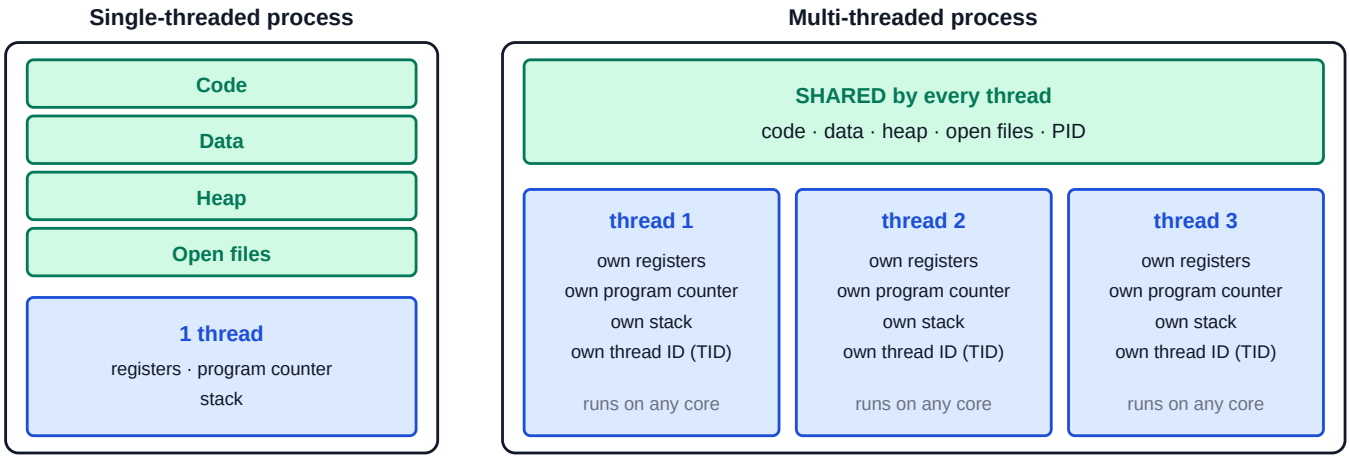
MEASURE

`time <command>`	**real** = wall clock, **user** + **sys** = CPU time. If real is much bigger, it spent time waiting (I/O-bound)
`top`	`wa` = % of time CPUs sat idle waiting on I/O
`iostat -x 1`	per-disk load (install `sysstat`)
`lpstat -o`	print jobs waiting in the spool (CUPS)

Processing type	How it runs
Batch	jobs run one after another (cron jobs, print queue)
Multiprogramming / multitasking	CPU switches between many programs
Real-time / embedded	must respond within a deadline (thermostat, car brakes)
Distributed	many machines over a network share the work

Spooling parks slow I/O (like a print job) on disk so the program can move on while a background service feeds the device. On Linux the print spool is `/var/spool/cups`.

7. Threads



Threads live inside a process. They share its memory, so they talk to each other cheaply. A bad thread can also corrupt or crash the whole process.

	Process	Thread
Memory	its own, isolated	shared with its process
Cost to create	heavy (`fork()`)	light (`pthread_create()`)
If it crashes	others keep running	can take the whole process down
Talks to others by	pipes, sockets, shared memory	just reading shared variables

User-level threads	Kernel-level threads
managed by a library inside the app; the kernel sees one process	managed and scheduled by the OS
fast to create; one blocking call can freeze them all	one can block while others keep running

Linux `pthreads` are kernel-level: each thread is its own scheduled task with its own TID (shown as LWP in `ps`).

SEE THREADS

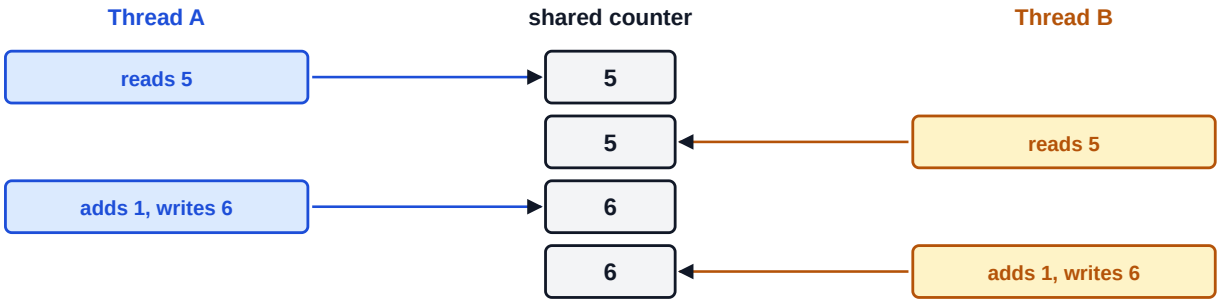
`ps -eLf`	every thread: LWP = thread ID, NLWP = thread count
`ps -T -p <PID>`	threads of one process
`top -H -p <PID>`	live view per thread
`ls /proc/<PID>/task`	one folder per thread
`grep Threads /proc/<PID>/status`	thread count

HARDWARE PARALLELISM

`nproc`	how many CPUs you can use
`lscpu | grep -E 'Socket|Core|Thread'`	Thread(s) per core = 2 means Hyper-Threading / SMT is on

Concurrency = taking turns fast enough to look simultaneous (even on one core). **Parallelism** = really at the same time on separate cores.

8. Concurrency problems: races and deadlocks



Two increments, but the counter went 5 → 6, not 7. One update was lost because both read before either wrote.

Fix: a lock (mutex) so only one thread can do read-add-write at a time.

A **race condition**: the result depends on which thread gets there first. This is the "lost update".

SEE A RACE ON YOUR OWN VM (TESTED)

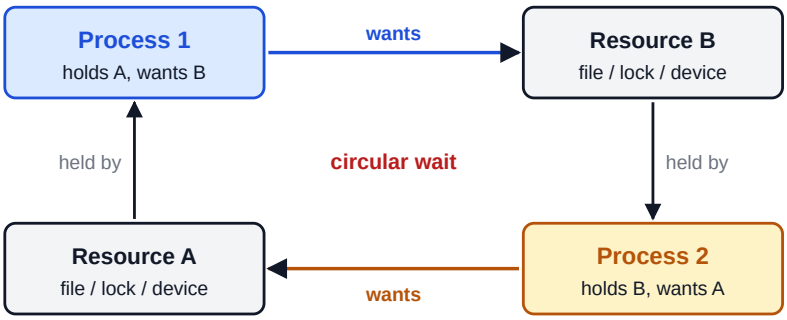
```
# no lock: two loops race on one file (expect far less than 2000)
echo 0 > count
inc() { for i in $(seq 1000); do n=$(cat count); echo $((n+1)) > count; done; }
inc & inc & wait; cat count

# with a lock: flock lets one loop in at a time (prints 2000)
echo 0 > count
inc() { for i in $(seq 1000); do
  flock count.lock -c 'n=$(cat count); echo $((n+1)) > count'; done; }
inc & inc & wait; cat count
```

Tool	What it does
Lock / mutex	one thread at a time in the critical section
Semaphore	a counter: up to N users at once
Atomic operation	read-add-write done as one uninterruptible step
Condition variable	wait until another thread signals "ready"

WHO HOLDS WHAT?

`cat /proc/locks`	file locks currently held
`lsof <file>` / `fuser -v <file>`	which processes have a file open



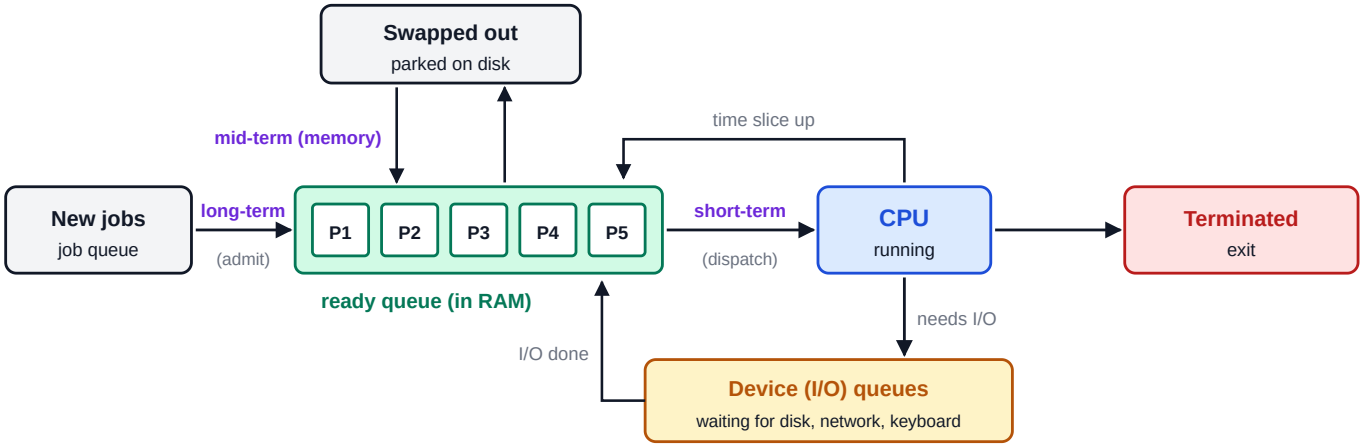
All four must be true for a deadlock:

1. Mutual exclusion: one user at a time
2. Hold and wait: keep A while asking for B
3. No preemption: nobody can take A away
4. Circular wait: P1 → P2 → P1

Break any one and it can't happen
(e.g. always lock A before B).

A **deadlock**: each process holds what the other needs, so both wait forever.

9. Schedulers and queues



Every arrow is a PCB moving from one queue to another. The dispatcher does the actual context switch.

Long-term scheduler: which new jobs get into memory. **Short-term** (CPU) scheduler: which ready process runs next, many times a second. **Mid-term** scheduler: swaps processes out to disk and back when memory is tight.

THE SCHEDULER DECIDES WHEN A PROCESS

1. goes from running to waiting (asked for I/O)
2. goes from running to ready (interrupt or time slice up)
3. goes from waiting to ready (I/O finished)
4. terminates

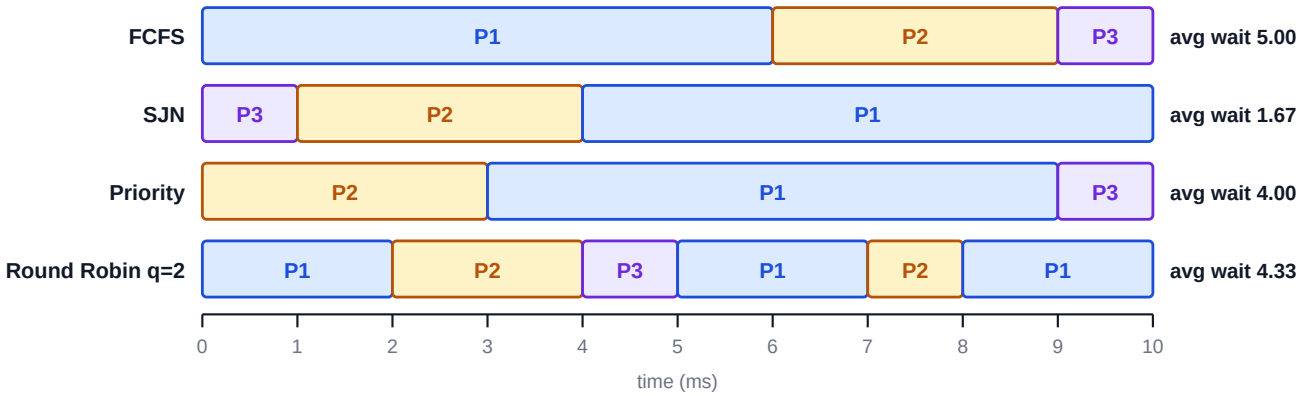
Only 1 and 4 happen with **non-preemptive** scheduling: the process gives up the CPU itself. **Preemptive** scheduling also uses 2 and 3: the OS can take the CPU away.

	Preemptive	Non-preemptive
Can the OS interrupt?	yes	no, runs until it waits or ends
Good for	desktops, servers, real-time, anything interactive	simple batch systems
Risk	more context-switch overhead	one long job blocks everyone
Examples	Round Robin, preemptive Priority, Linux	FCFS, SJN

Goals: fairness (everyone gets a turn), predictability (steady response time), scalability (still works with many processes).

10. Scheduling algorithms, worked example

Three made-up processes all arrive at time 0, in the order P1, P2, P3. CPU burst times: **P1 = 6 ms**, **P2 = 3 ms**, **P3 = 1 ms**. For Priority, P2 is most important, then P1, then P3.



Gantt charts: who has the CPU at each millisecond.

Algorithm	Rule	Watch out
FCFS	first come, first served	short jobs stuck behind long ones (convoy effect)
SJN / SJF	shortest burst first	lowest average wait, but long jobs can starve; burst times must be guessed
Priority	most important first	low priority can starve; fix with **aging** (raise priority while waiting)
Round Robin	everyone gets a quantum, then goes to the back	quantum too small = too many switches; too big = acts like FCFS

FORMULAS

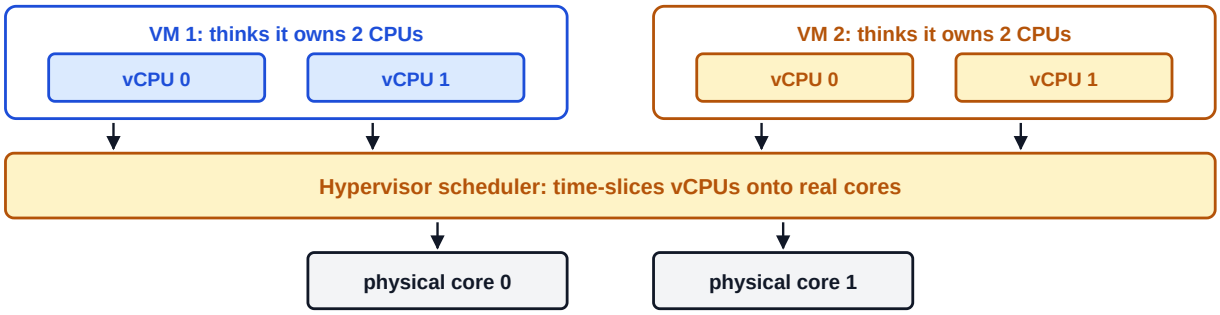
Turnaround = finish time − arrival time
Waiting = turnaround − burst time
Response = first time on CPU − arrival time

Round Robin check: P1 finishes at 10 (wait 10 − 6 = 4), P2 at 8 (wait 5), P3 at 5 (wait 4). Average = 13 / 3 = **4.33**.

LINUX SCHEDULING COMMANDS

`chrt -p <PID>`	a process's scheduling policy and priority
`chrt -m`	priority ranges for each policy
`sudo chrt -r 10 <cmd>`	run as real-time Round Robin, priority 10
`sudo chrt -f 10 <cmd>`	run as real-time FIFO (FCFS-style)
`cat /proc/sys/kernel/sched_rr_timeslice_ms`	Round Robin quantum (default 100)
`nice -n 10 <cmd>` / `sudo renice -5 -p <PID>`	normal priority: −20 (highest) to 19 (lowest)
`ps -eo pid,cls,ni,pri,cmd`	CLS: TS = normal, FF = FIFO, RR = Round Robin
`taskset -c 0 <cmd>`	pin a process to CPU core 0

Normal Linux processes (`SCHED_OTHER`) use a fair scheduler: CFS, replaced by EEVDF in kernel 6.6. Real-time ones (`SCHED_FIFO` , `SCHED_RR` , priorities 1 to 99) always run before normal ones.



Virtual machines: the hypervisor schedules each VM's virtual CPUs onto the real cores. Inside a VM, `st` (steal) in `top` or `vmstat` is time the hypervisor gave your vCPU to someone else.

11. Security: processes and threads

Threat	What happens	Defense / lab check
Race condition / TOCTOU	attacker swaps a file between the program's check and its use (classic `/tmp` symlink trick)	locks, `mktemp` ; `sysctl fs.protected_symlinks` should be 1
Deadlock (on purpose)	service hangs: availability hit	lock ordering, timeouts, monitoring
Resource exhaustion	runaway threads or a fork bomb eat CPU, memory or the process table (DoS)	`ulimit -u` , `/etc/security/limits.conf` , systemd `TasksMax`
Stack buffer overflow	overwrites a return address to run attacker code	bounds checks, ASLR (`randomize_va_space` = 2), NX, stack canaries

Thread / code injection	malware writes into another process and starts a thread there	`cat /proc/sys/kernel/yama/ptrace_scope` (1 or higher = restricted), EDR
Privilege escalation	bug in a high-privilege process or thread gives an attacker its rights	least privilege, patching, drop privileges early
Data leak between threads	shared memory exposes secrets; non-thread-safe libraries corrupt data	thread-safe libraries, isolate secrets in separate processes
Timing attack	how long something takes leaks information (e.g. password compare)	constant-time comparisons

HUNT

`ps -eo pid,user,%cpu,%mem,cmd --sort=-%cpu | head`	top CPU eaters (spot runaways or miners)
`ps -eo user= | sort | uniq -c | sort -rn`	process count per user (fork bomb check)
`ulimit -u`	max processes for your user
`ls -l /proc/<PID>/exe`	the real program behind a process (catches renamed malware)

12. Common mix-ups

- **PCB** = process control block.
- **Text section** holds the program's instructions. The program counter and registers live in the CPU, and are saved into the **PCB** on a context switch.
- **CPU modes** are called **kernel mode** and **user mode**.
- **64-bit memory:** 16 EB (2^{64} bytes) is the theoretical ceiling. Today's x86-64 CPUs use 48-bit (256 TB) or 57-bit virtual addresses.
- **32-bit programs on 64-bit** only run if 32-bit libraries are installed. macOS dropped 32-bit apps entirely in 10.15 Catalina.
- **DEP and ASLR** exist on 32-bit too. 64-bit makes ASLR far stronger because there are vastly more places to hide things.
- **One process at a time** is per **core**. A 4-core CPU really does run 4 at once.
- **Threads** share the process's memory and files, but each has its own registers, program counter and stack (a thread control block). In Linux, each thread is its own `task_struct` with its own TID.
- **Deadlock** needs all four: mutual exclusion, hold and wait, no preemption, circular wait. Break any one to prevent it.
- **CPU burst** = how long a process computes before it has to wait (usually for I/O). **I/O burst** = that wait.
- **New processes** start as **not running** (ready), waiting in the queue until the dispatcher picks them.
- **Round Robin** is always preemptive (that is the time quantum). Priority scheduling can be either.